Documento de Aprendizaje Continuo (AC)

AuraBot: Robot Aspiradora Autónomo con Yocto

Jose Bernardo Barquero 2023150476
Alexander Montero 2023166058
Jimmy Feng 2023060347
Jose Edo Campos 2023135620

Instituto Tecnológico de Costa Rica
CE-1113 Sistemas Empotrados

14 de octubre de 2026

1. Introducción

El desarrollo de AuraBot requiere integrar conocimientos de programación en C, sistemas operativos, control de hardware, comunicación entre procesos y construcción de imágenes Linux. La complejidad del proyecto no se limita a implementar cada funcionalidad, sino que también incluye conseguir que todos los componentes funcionen de manera coordinada sobre una Raspberry Pi 4 con recursos y configuraciones específicos.

Durante el desarrollo se identificaron necesidades de aprendizaje relacionadas con Yocto, la compilación cruzada, la interacción con los GPIO, la generación de PWM por software y la integración de audio con el control del robot.

Este documento presenta las principales necesidades identificadas, las tecnologías estudiadas, las estrategias utilizadas para resolver los problemas y los criterios para evaluar la efectividad del aprendizaje.

2. AC1: Identificación de necesidades de aprendizaje

2.1. Integración de software con el sistema operativo

Una de las necesidades principales fue comprender cómo organizar una aplicación embebida para que sus componentes pudieran construirse, instalarse y ejecutarse de manera indepen-

diente.

En AuraBot no basta con compilar el programa principal. También es necesario generar las bibliotecas dinámicas, declarar sus dependencias, integrarlas en la imagen Yocto y configurar los servicios que permiten iniciar las aplicaciones automáticamente.

Por esta razón, fue necesario profundizar en el funcionamiento de CMake, las bibliotecas compartidas, las recetas BitBake y systemd.

2.2. Control de hardware desde Linux

El control de motores y sensores requirió estudiar cómo interactuar con los GPIO desde un programa en C. También fue necesario comprender las diferencias entre el PWM por hardware y el PWM generado mediante software.

Esta necesidad se relaciona directamente con la arquitectura de AuraBot, ya que el sistema debe controlar dos motores, interpretar las lecturas de los sensores infrarrojos y procesar los pulsos de los encoders Hall.

Además, se necesitó aprender a diagnosticar errores que pueden surgir al exportar un GPIO, configurar su dirección o realizar escrituras sobre las interfaces expuestas por el kernel.

2.3. Concurrencia y comunicación entre procesos

El robot debe coordinar la navegación, la comunicación remota y la reproducción de audio. Por ello, fue necesario estudiar mecanismos para separar las responsabilidades de los procesos sin perder la capacidad de controlar el estado general del sistema.

En el proyecto se utilizan hilos para determinadas tareas de control, comunicación IPC entre los clientes y el proceso principal, y una FIFO para enviar órdenes al servidor de audio.

También fue necesario considerar qué ocurre cuando dos clientes intentan controlar el robot o cuando un servicio deja de funcionar.

2.4. Diagnóstico y pruebas

Otra necesidad de aprendizaje fue diferenciar los errores de compilación de los errores de configuración del sistema operativo y de los problemas físicos del hardware.

Por ejemplo, que una función de configuración PWM retorne correctamente no garantiza que la señal continúe alternando en el GPIO. Del mismo modo, una compilación satisfactoria no demuestra por sí sola que el binario funcione correctamente en la Raspberry Pi.

Esto motivó el uso de pruebas unitarias, registros de ejecución, herramientas de diagnóstico y verificaciones específicas sobre la plataforma objetivo.

3. AC2: Tecnologías y conocimientos adquiridos

3.1. Yocto y BitBake

Yocto permite construir una imagen Linux adaptada a las necesidades de AuraBot. El proyecto utiliza la capa `meta-ce1113`, con recetas separadas para las bibliotecas, el hardware, las aplicaciones y la configuración del sistema.

El aprendizaje de BitBake fue necesario para comprender cómo declarar dependencias, instalar los artefactos generados por CMake y componer la imagen `ce1113-p1` mediante grupos de paquetes.

Esta organización también permite mantener las aplicaciones de prueba separadas de la imagen destinada al robot.

3.2. Bibliotecas dinámicas y CMake

El proyecto utiliza bibliotecas compartidas para separar funcionalidades y evitar que cada aplicación implemente de nuevo el acceso al hardware.

Entre los componentes desarrollados se encuentran `libgpio`, `libpwm`, `libaudio` y las bibliotecas de control y comunicación de AuraBot.

CMake permite compilar e integrar estos componentes, mientras que las recetas de Yocto declaran las dependencias necesarias para construirlos e instalarlos en el sistema final.

3.3. PWM por software y acceso a GPIO

El conflicto entre el PWM de hardware de la Raspberry Pi 4 y su salida de audio analógica motivó el estudio de una implementación alternativa.

La biblioteca `libpwm` genera las transiciones de la señal mediante hilos y temporización, mientras que `libgpio` se encarga del acceso a los pines.

Esta separación permitió estudiar aspectos de concurrencia, temporización, gestión de errores y diferencias entre las interfaces de Linux y el comportamiento esperado del hardware.

3.4. Audio con ALSA y mpg123

AuraBot utiliza `mpg123` para reproducir archivos MP3 y ALSA para gestionar la salida de audio.

La biblioteca `libaudio.so` encapsula las operaciones de reproducción. Un servidor independiente recibe comandos por la FIFO `/run/aurabot-audio/control`, lo que permite desacoplar la interfaz web del proceso que gestiona el reproductor.

Además, el proyecto utiliza la partición `AURA_AUDIO` para almacenar las canciones y la lista de reproducción en `/media/audio`.

3.5. IPC, servicios y comunicación web

El proceso principal `aurabot` administra el estado del robot y ofrece comunicación local mediante IPC. La biblioteca API permite que otros componentes soliciten operaciones sin acceder directamente a los GPIO.

Por su parte, la interfaz web utiliza CGI en C y tecnologías web convencionales. El sistema incorpora autenticación, control de sesiones y mecanismos para evitar que múltiples clientes conduzcan simultáneamente el robot.

El uso de systemd permite configurar el inicio automático y la recuperación de servicios ante determinados fallos.

4. AC3: Estrategias de aprendizaje implementadas

4.1. Investigación mediante documentación técnica

Se utilizaron documentos técnicos del propio proyecto, documentación de las herramientas y registros de los errores para comprender la función de cada componente.

Esta estrategia resulta especialmente importante en Yocto, donde una aplicación puede compilar correctamente de forma aislada, pero fallar al integrarse en la imagen por una dependencia ausente, una receta incompleta o un problema de configuración del sistema.

4.2. Desarrollo incremental

El trabajo se organizó en componentes con responsabilidades separadas. Por ejemplo, el acceso a GPIO se mantiene en `libgpio`, la generación de PWM en `libpwm` y el control general en `aurabot`.

Esta división permite verificar las funciones de bajo nivel antes de incorporarlas al sistema completo. También facilita determinar si un error se encuentra en la biblioteca, en el controlador o en la configuración de Yocto.

4.3. Diagnóstico de problemas de PWM

La implementación de PWM por software presentó dificultades relacionadas con el acceso a los GPIO y la temporización.

Entre los aspectos estudiados y corregidos en el código se encuentran:

- La necesidad de escribir correctamente los valores sobre las interfaces GPIO de Linux, incluyendo el salto de línea esperado por las operaciones de escritura.
- La sincronización de la inicialización de los GPIO, debido a que la exportación de un pin puede completarse de forma asíncrona.
- El uso de enteros de 64 bits para calcular períodos y tiempos de PWM, evitando desbordamientos en operaciones que pueden superar el rango de un entero de 32 bits.
- La gestión de interrupciones de `nanosleep` mediante `EINTR`, para completar el tiempo de espera restante cuando corresponda.
- La comprobación de errores y la recuperación ante fallos transitorios de escritura.

Estos cambios ilustran la importancia de considerar las particularidades de la plataforma objetivo y no limitarse a comprobar que el programa compila.

4.4. Pruebas con hardware simulado

El repositorio incluye pruebas que sustituyen determinadas operaciones físicas por implementaciones simuladas.

En particular, se dispone de pruebas para la lógica de PWM mediante un GPIO falso, así como pruebas del núcleo, la máquina de estados y el mapa.

Este enfoque permite comprobar comportamientos esperados sin necesitar que todos los componentes estén conectados. Una vez superadas estas pruebas, las funciones que dependen del hardware deben validarse por separado en la Raspberry Pi.

4.5. Automatización de la construcción

Se utilizan scripts para organizar la construcción y verificación del repositorio, además de una configuración de integración continua que valida la estructura del proyecto y compila el código C mediante CMake.

Estas herramientas ayudan a detectar errores antes de integrar cambios en el sistema. La construcción completa de Yocto y la verificación de la imagen final siguen requiriendo las comprobaciones correspondientes en el entorno de desarrollo.

5. AC4: Evaluación crítica del aprendizaje

5.1. Efectividad de las estrategias

El desarrollo modular facilita aislar problemas y evita tener que depurar toda la aplicación cuando falla una funcionalidad específica. Las pruebas con hardware simulado también

permiten repetir escenarios de error y comprobar el comportamiento del código de manera controlada.

Por otra parte, los scripts de construcción y validación reducen la dependencia de procedimientos manuales y ayudan a conservar una estructura consistente en el repositorio.

La investigación de problemas concretos también permite profundizar en aspectos que no siempre son evidentes durante la programación convencional, como la temporización de GPIO, la carga de bibliotecas dinámicas y la interacción entre los servicios del sistema operativo.

5.2. Limitaciones identificadas

Las pruebas locales no reproducen completamente las condiciones de una Raspberry Pi en funcionamiento. La planificación de hilos, el comportamiento de los periféricos, el cableado y la alimentación pueden introducir problemas que no aparecen en una simulación.

De forma similar, la compilación satisfactoria de una biblioteca no garantiza que la receta de Yocto esté completa ni que el servicio se inicie correctamente después de arrancar desde la tarjeta SD.

Por esta razón, las pruebas automatizadas deben complementarse con verificaciones en la plataforma real, especialmente para el control de motores, la reproducción simultánea de audio, el inicio de los servicios y las métricas de consumo de recursos.

5.3. Evidencias y evaluación final

La siguiente tabla permite relacionar el aprendizaje con evidencias concretas del proyecto.

Área	**Evidencia**	**Evaluación**
Yocto y BitBake	Recetas de `meta-ce1113`, grupos de paquetes y configuración de `ce1113-p1`.	Verificar la construcción final y conservar el log real de compilación cruzada.
GPIO y PWM	Bibliotecas `libgpio` y `libpwm`, junto con las pruebas de GPIO simulado.	Revisar los resultados automatizados y medir las señales en el hardware real.
Audio	`libaudio.so`, servidor de audio, FIFO y configuración ALSA.	Comprobar reproducción, pausa, volumen y concurrencia con la navegación.

Control del robot	Módulos de navegación, máquina de estados, odometría y mapa.	Revisar las pruebas de software y contrastarlas con un recorrido físico.
Interfaz web	Código CGI, autenticación, control de sesiones y mapa web.	Ejecutar pruebas de integración y verificar el funcionamiento desde otro dispositivo.
Eficiencia y servicios	Herramientas de métricas y unidades de systemd.	Registrar valores reales de rootfs, arranque, CPU y RAM.

La evaluación definitiva debe completarse con los resultados observados en el sistema final. En particular, se debe indicar qué pruebas se ejecutaron satisfactoriamente, cuáles continúan pendientes y qué cambios fueron necesarios para corregir los problemas encontrados.

6. Conclusiones

El desarrollo de AuraBot permitió abordar problemas que trascienden la programación aislada de una aplicación. La construcción de una imagen Linux personalizada, la separación en bibliotecas dinámicas y la integración de servicios requieren comprender las relaciones entre el software, el sistema operativo y el hardware.

Los problemas relacionados con el acceso a GPIO, la temporización del PWM y la integración del audio muestran la importancia de combinar la investigación técnica con la experimentación y el diagnóstico sistemático.

Finalmente, las pruebas automatizadas, la construcción incremental y la documentación de los errores constituyen estrategias útiles para mejorar la reproducibilidad y la calidad del sistema. La validación física y las mediciones finales permitirán determinar qué tan efectivas fueron estas estrategias en el funcionamiento completo del robot.

Este documento fue generado con AI y posteriormente revisado y edita de forma manual